Descriptor-Aligned Zobrist Hashing for ReSolvitaire

Revision to the Cache Redesign

Ian Gent AI Assistant (Claude, Anthropic)

March 2026

AI Generation Statement: *This document was produced collaboratively. The central insight—that the Zobrist hash should be aligned with the per-card descriptor values from the payload, rather than with pile roles and positions—is Gent's. The AI contributed the analysis of its consequences: elimination of the two-layer combining scheme, simplification of the Zobrist table to ∼4 KB, and the merging of hash and payload maintenance into a single code path.*

1 Context

The *ReSolvitaire Cache Redesign* document [1] specifies two parallel structures maintained incrementally during search:

1. A **64-bit Zobrist hash** for cache table indexing, using a two-layer scheme: per-pile hashes via XOR (Layer 1) combined into a global hash via XOR for non-interchangeable piles and modular addition for interchangeable piles (Layer 2) [3, 4].
2. A **32-byte compact payload** of per-card 4-bit descriptors for zero-false-positive verification on cache hits.

The Zobrist hash required a key table indexed by (card, pile role, position within pile), with a two-layer combining strategy to achieve symmetry invariance. The payload used descriptor values {0=Starting, 1=In_Hole, 2=Root, 3=In_Cell, 4–7=Parent_0–Parent_3} which are inherently pile-index-free.

This document describes a revision that aligns the Zobrist hash with the payload descriptors, substantially simplifying the architecture.

2 The Insight

The payload descriptors already capture, for each card, everything the Zobrist hash needs to distinguish. A card's descriptor encodes its "situation" without reference to pile indices, pile positions, or pile roles. Two game states are equivalent (up to interchangeable-pile permutation) if and only if every card has the same descriptor in both states, the foundation tops are equal, and the waste pointer is equal.

Therefore, the Zobrist hash should hash *descriptors*, not *pile memberships*. The key table becomes $Z[\text{card\_id}][\text{descriptor\_value}]$: one random 64-bit value for each (card, descriptor) pair. The global hash is simply:

$$H = \bigoplus_{c=0}^{51} Z[c][\text{desc}(c)] \quad \oplus \quad Z_{\text{found}}[\text{suit}][\text{top\_rank}] \quad \oplus \quad Z_{\text{waste}}[\text{ptr}]$$

where $\oplus$ denotes XOR.

3 Consequences

3.1 Elimination of the Two-Layer Scheme

The original two-layer design was needed because pile-role-position Zobrist keys depend on pile indices: moving a card from pile 3 to pile 5 changes the keys. Permuting interchangeable piles changes every card's keys, so the Layer 2 additive combining was introduced to make the global hash invariant under such permutations [4, 5].

With descriptor-aligned keys, **no card's key depends on any pile index**. "PARENT_0" means "built on a specific parent card," identified by the parent's identity, not by which pile the parent is in. Permuting interchangeable piles changes no card's descriptor, hence changes no key, hence changes the hash by zero.

The two-layer scheme—per-pile hashes, additive combining, the distinction between interchangeable and non-interchangeable piles—is entirely unnecessary. The hash is a flat XOR of 52 per-card contributions plus header contributions. Symmetry invariance is automatic.

3.2 Dramatic Reduction in Zobrist Table Size

Design	**Entries**	**Size**
Original (card × role × position)	$52 \times 7 \times 104 = 37{,}856$	∼295 KB
Position-only (card × position)	$52 \times 104 = 5{,}408$	∼42 KB
Descriptor-aligned (card × descriptor)	$52 \times 8 = 416$	∼3.3 KB
+ foundation tops	$4 \times 14 = 56$	448 B
+ waste pointer	64	512 B
Total (descriptor-aligned)	**536**	**∼4.3 KB**

The full table fits in L1 data cache on any modern CPU (≥32 KB per core). Every lookup is a single array index with no computation. Initialisation is ∼500 calls to `mt19937_64`—sub-microsecond.

3.3 Unified Incremental Update

Because the hash and payload are keyed by the same event (a card's descriptor changing), they are updated in the same code path. When a card moves and its descriptor changes from d_{old} to d_{new}:

```
payload.set_descriptor(card_id, d_new);
zobrist ^= Z[card_id][d_old] ^ Z[card_id][d_new];
```

One nibble write, two XOR operations. For foundation changes:

```
payload.set_foundation(suit, new_top);
zobrist ^= Z_found[suit][old_top] ^ Z_found[suit][new_top];
```

For waste pointer changes:

```
payload.set_waste_ptr(new_ptr);
zobrist ^= Z_waste[old_ptr] ^ Z_waste[new_ptr];
```

No per-pile hashes to maintain. No addition/subtraction for interchangeable piles. No Layer 1/Layer 2 bookkeeping. x

4 The Starting_Face_Up Descriptor

The original payload specification uses descriptor value 0 (STARTING) for all cards in their initial deal position, whether face-down or face-up. This creates an ambiguity: a card originally dealt face-down that has been revealed (turned face-up by a move removing the card above it) but not itself moved is still STARTING—yet the game state has changed (different moves are now legal, and the card's face-down status is different).

To resolve this, we add descriptor value 1, immediately after STARTING:

Value	Name	Meaning
0	STARTING	Card in original deal position *and* in original face-up/face-down state. Also used for cards on foundations (descriptor ignored).
1	**Starting_Face_Up**	**Card originally dealt face-down, now revealed (turned face-up), but not moved from its original pile position.**
2	ROOT	Bottom face-up card of a tableau pile, moved from starting position.
3	IN_CELL	Card in a free cell.
4	PARENT_0	Built on first legal parent.
5	PARENT_1	Built on second legal parent.
6	PARENT_2	Built on third legal parent.
7	PARENT_3	Built on fourth legal parent.
8	IN_HOLE	Card played to hole (hole games only).
9–15	RESERVED	Zero. For future use (two-deck parents, streamliners).

4.1 Transition Rules

Reveal: When a move has `reveal_move = true`, the newly revealed card transitions from STARTING (0) to STARTING_FACE_UP (1). Both the payload nibble and the Zobrist hash are updated.

Move after reveal: If a STARTING_FACE_UP card is subsequently moved (to a parent, cell, foundation, etc.), it transitions to the appropriate descriptor (PARENT_i, IN_CELL, etc.).

Undo reveal: On `undo_move` of a reveal move, the card transitions from STARTING_FACE_UP back to STARTING.

Cards never face-down: In games with `face_up = ALL` (no face-down cards), STARTING_FACE_UP is never used. All starting cards are STARTING and transition directly to their moved descriptor.

4.2 Correctness

With this descriptor, STARTING now unambiguously means "in original position *and* in original face-up/face-down state." Two states differing only in whether a card has been revealed produce different payloads (0 vs. 1 for that card) and different Zobrist hashes ($Z[c][0]$ vs. $Z[c][1]$). This eliminates the face-down ambiguity entirely.

The cost is one additional nibble write and two additional XOR operations per reveal move—negligible, since reveal moves are a subset of regular moves that already require descriptor updates for the moved card.

5 Zobrist Table Specification

5.1 Key Tables

Three static arrays, initialised once at program start with a fixed-seed `mt19937_64`:

Table	Dimensions	Entries
`Z_card[52][16]`	card ID × descriptor value	832
`Z_found[4][14]`	suit × top rank (0–13)	56
`Z_waste[64]`	waste pointer (0–63)	64

Notes:

- `Z_card` has 16 columns to accommodate all descriptor values: STARTING (0), STARTING_FACE_UP (1), ROOT (2), IN_CELL (3), PARENT_0–PARENT_3 (4–7), IN_HOLE (8), and reserved values 9–15 for future extensions (two-deck parents, streamliners). Columns 0–8 are used for single-deck games; 9–15 are reserved. Total: 832 entries × 8 bytes = 6.5 KB.
- `Z_found` covers all single-deck foundation states. Rank 0 = empty; rank 1 = Ace; ...; rank 13 = King. For hole games, this table is unused (the hole top is captured by a card having descriptor IN_HOLE, and the hole-top field in the payload has its own hash entry).
- `Z_waste` covers waste pointer values 0–63. For games without stock, the waste pointer is always 0 and contributes a fixed hash value (effectively a no-op after XOR with the initial hash).

For hole games, an additional small table is needed:

`Z_hole_top[52]`	hole top card ID	52

This is mutually exclusive with `Z_found` (a game has foundations or a hole, never both).

5.2 Hash Computation

Initialisation. At game start, all cards have descriptor STARTING (0):

$$H_0 = \bigoplus_{c=0}^{51} Z_{\text{card}}[c][0] \;\oplus\; \bigoplus_{s=0}^{3} Z_{\text{found}}[s][\text{init\_top}(s)] \;\oplus\; Z_{\text{waste}}[0]$$

For games with pre-filled foundations (`foundations_init_cards` = ONE or ALL), init_top(s) reflects the initial top rank and the corresponding cards' descriptors are set to STARTING (they are on foundations and their descriptors are ignored in comparison, but zeroed for `memcmp` correctness).

Incremental update. On each `make_move`:

1. For each card whose descriptor changes ($d_{\text{old}} \to d_{\text{new}}$):
$$H \oplus= Z_{\text{card}}[c][d_{\text{old}}] \oplus Z_{\text{card}}[c][d_{\text{new}}]$$
2. If foundation top changes (suit s, rank $r_{\text{old}} \to r_{\text{new}}$):
$$H \oplus= Z_{\text{found}}[s][r_{\text{old}}] \oplus Z_{\text{found}}[s][r_{\text{new}}]$$
3. If waste pointer changes ($p_{\text{old}} \to p_{\text{new}}$):
$$H \oplus= Z_{\text{waste}}[p_{\text{old}}] \oplus Z_{\text{waste}}[p_{\text{new}}]$$
4. If hole top changes ($t_{\text{old}} \to t_{\text{new}}$):
$$H \oplus= Z_{\text{hole\_top}}[t_{\text{old}}] \oplus Z_{\text{hole\_top}}[t_{\text{new}}]$$

On `undo_move`: identical operations (XOR is self-inverse).

Cost per move. Most moves change one card's descriptor and at most one header field: 2 XOR operations and 1 nibble write. A move to foundations changes one descriptor + one foundation top: 4 XOR operations and 2 field writes. A stock-to-all-tableau deal changes T descriptors + waste pointer: $2T + 2$ XOR operations. All operations are single array lookups followed by XOR—no multiplication, no addition, no branching.

6 Hash Quality

With 52 cards each contributing one of 9 (or up to 16) independent random 64-bit keys, the hash has $9^{52} \approx 10^{49}$ possible values (far exceeding $2^{64} \approx 1.8 \times 10^{19}$, meaning the key space is densely covered). The standard Zobrist collision analysis applies [3]: for N cached states with a 64-bit hash, the expected number of collisions is $\binom{N}{2}/2^{64}$. At $N = 10^9$: $\sim$0.027 expected collisions. Since collisions are resolved by payload comparison (zero false positives), this affects only cache-indexing efficiency, not correctness.

The foundation and waste contributions add independent hash components, improving distribution further.

7 Payload Byte Layout

The 32-byte compact state payload is laid out as follows:

Field	Offset	Size	Description
Occupied	byte 0	1 byte	0 = empty slot; nonzero = occupied
Depth	bytes 1–2	2 bytes	16-bit search depth (excluded from comparison)
Foundations	bytes 3–4	2 bytes	4 foundation top ranks (4 bits each) OR hole-top card ID (6 bits + padding)
Waste-ptr	byte 5	1 byte	Waste pointer (0–63; 0 if no stock)
Descriptors	bytes 6–31	26 bytes	52 per-card descriptors (4 bits each)

Total: $1 + 2 + 2 + 1 + 26 = 32$ bytes.

Bytes 0–2 are metadata, excluded from state comparison. Cache hit verification compares bytes 3–31 only (29 bytes):

```
bool match = (memcmp(data + 3, other.data + 3, 29) == 0);
```

Card descriptor nibble access for card c ($0 \leq c \leq 51$):

```
byte_idx = 6 + (c / 2);
// Even c: low nibble.  Odd c: high nibble.
```

8 Impact on the Implementation Plan

This revision affects the implementation plan [2] as follows:

Milestones 0 and 1: Unchanged. Repository setup, baseline, and abstract cache interface remain valid.

Milestones 2 and 3 merge: The Zobrist hash and compact payload are so tightly coupled that they should be implemented together as a single milestone. The per-pile-hash infrastructure from the original Milestone 2 is replaced by the descriptor-aligned scheme described here. The key tables, incremental update logic, and payload maintenance share the same `make_move`/`undo_move` integration points.

Milestones 4 onward: Unchanged. The flat cache, solver wiring, verification, pile-ordering removal, and benchmarking proceed as planned. The flat cache's `insert` and `contains` methods use the Zobrist hash for indexing and the payload for verification, regardless of how the hash was computed.

Milestone 7 (pile ordering removal): This becomes even simpler. The original plan noted that pile ordering affected both the cache encoding (now irrelevant—encoding is descriptor-based) and move generation order. With the descriptor-aligned hash, there is no cache-related reason to maintain pile ordering. The only remaining reason is move generation order, which is a performance tuning question, not a correctness one.

9 Summary

	Original design	**Revised design**
Zobrist table	$52 \times 7 \times 104$ entries, ∼295 KB	$52 \times 16 + 56 + 64 + 52$ entries, ∼8 KB
Combining scheme	Two-layer: per-pile XOR, global XOR + addition	Flat XOR of per-card keys
Symmetry handling	Additive combining for interchangeable piles	Automatic (descriptors are pile-free)
Update per move	Per-pile hash update + global hash update + payload nibble write	Hash XOR + payload nibble write (unified)
Per-pile hash state	One `uint64_t` per pile	None
Code complexity	Separate hash and payload maintenance paths	Single unified path

The descriptor-aligned Zobrist hash is simpler, smaller, faster, and automatically symmetry-invariant. It achieves these properties by recognising that the payload's per-card descriptors already provide the minimal distinguishing information for each card's situation, and aligning the hash directly with that information.

References

[1] Gent, I. P. & AI Assistant (2026). ReSolvitaire Cache Redesign: Architecture and Payload Specification. Unpublished working document.

[2] Gent, I. P. & AI Assistant (2026). ReSolvitaire Cache Refactor: Implementation Plan. Unpublished working document.

[3] Zobrist, A. L. (1970). *A New Hashing Method with Application for Game Playing.* Tech. Report 88, CS Dept., University of Wisconsin–Madison.

[4] Clarke, D., Devadas, S., van Dijk, M., Gassend, B., & Suh, G. E. (2003). Incremental Multiset Hash Functions and Their Application to Memory Integrity Checking. *ASIACRYPT 2003*, LNCS 2894, pp. 188–207.

[5] Kun, J. (2021). Group Actions and Hashing Unordered Multisets. https://www.jeremykun.com/2021/10/14/group-actions-and-hashing-unordered-multisets/